

EXPERIMENT-05

Aim:

To demonstrate the performance of a linear regression by using chosen database with python code

Algorithm:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


# 1. Load chosen dataset

data = load_diabetes()

X = data.data

y = data.target


# 2. Split dataset into training and testing data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)


# 3. Create and train Linear Regression model

model = LinearRegression()

model.fit(X_train, y_train)
```

4. Predict test values

```
y_pred = model.predict(X_test)
```

5. Evaluate model performance

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("Linear Regression Performance")
```

```
print("-----")
```

```
print("Mean Absolute Error (MAE):", round(mae, 2))
```

```
print("Mean Squared Error (MSE):", round(mse, 2))
```

```
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
```

```
print("R2 Score:", round(r2, 2))
```

6. Plot actual vs predicted values

```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(y_test, y_pred, color="blue", alpha=0.7)
```

```
plt.plot(
```

```
    [y_test.min(), y_test.max()],
```

```
    [y_test.min(), y_test.max()],
```

```
    color="red",
```

```
    linewidth=2,
```

```
    label="Perfect Prediction"
```

```
)
```

```
plt.title("Linear Regression: Actual vs Predicted Values")
plt.xlabel("Actual Diabetes Progression")
plt.ylabel("Predicted Diabetes Progression")
plt.legend()
plt.grid(True)
plt.show()
```

CODE:

```
▶ # Linear Regression Performance Analysis
# Dataset: Diabetes dataset from scikit-learn

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Load chosen dataset
data = load_diabetes()
X = data.data
y = data.target

# 2. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# 3. Create and train Linear Regression model
```

OUTPUT

...

